Hard Problems

Design and Analysis of Algorithms

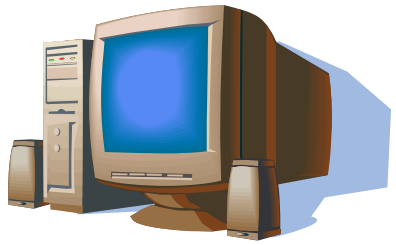
Brian C. Dean

School of Computing

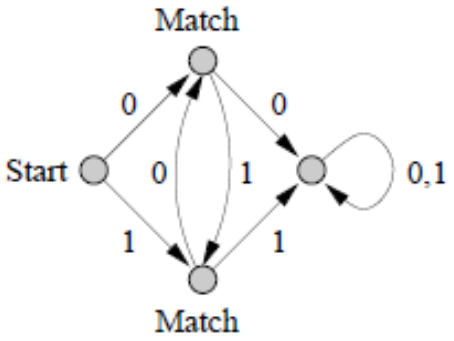
Clemson University

Aside: A Brief Introduction to the Theory of Computation

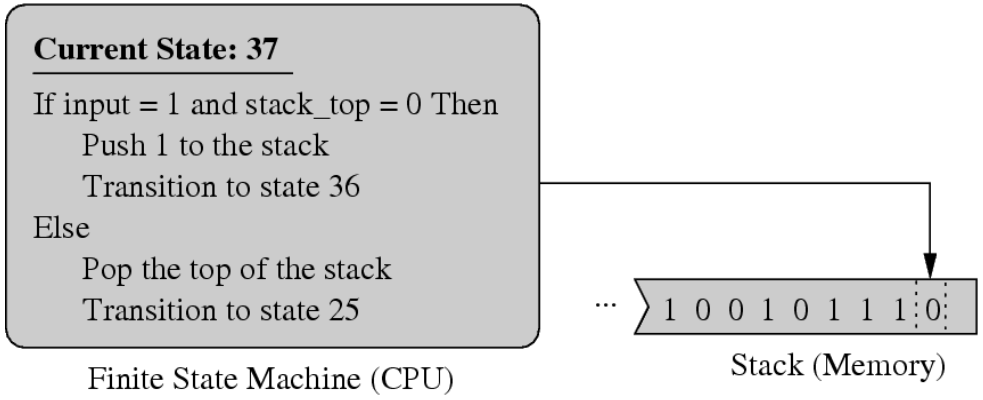
What is a computer?



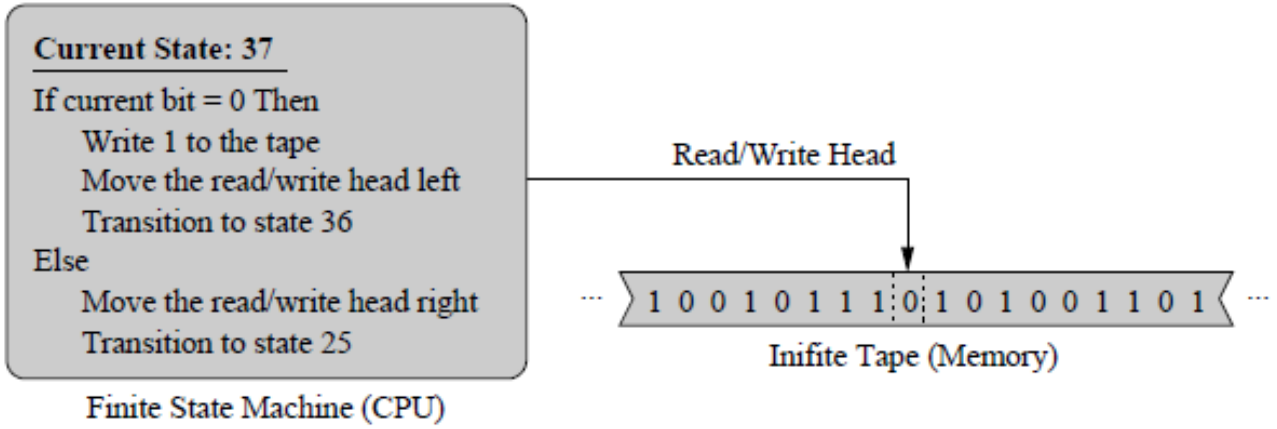
Different Models of Computing Machines



Finite-state automation
(recognizes binary strings of the form (0(10))|(1(01)*))*



Push-down automation



Turing machine

Different Models of Computing Machines

- Simple models of computation:
  - Finite state automata
  - Pushdown automata
  - Turing machines
- “Non-regular” problems cannot be solved by FSAs, but these might be solvable with PDAs.
- “Non-context-free” problems cannot be solved by PDAs, but these might be solvable with a Turing machine.
- “Undecidable” problems cannot be solved by Turing machines, and these actually cannot be solved by *any* computing machine!
  - **Alan Turing (1930s):** no computing machine is more powerful than a Turing machine in terms of the set of problems it can solve.
  - The **halting problem** is a famous undecidable problem.

From Computability to Complexity Theory

- Turing's work helped define what problems can and cannot be solved by algorithms.
- The next question is what problems can be solved **efficiently** by algorithms in various models of computation.
- Edmonds (1960s): "Efficiently" = "in polynomial runing time". I.e., $O(n^2)$ or $O(n^3)$, not $O(2^n)$.
- One of our main jobs as algorithm designers is to figure out if problems can be solved efficiently…

Relating Problem Difficulty with Reductions

- Recall that element uniqueness (testing if a set of n elements are all distinct) has an Ω(n log n) worst-case lower bound in the comparison model.
- Similarly, finding the most frequently occurring element must also take Ω(n log n) time in the worst case in the comparison model, since we can reduce element uniqueness to this problem.

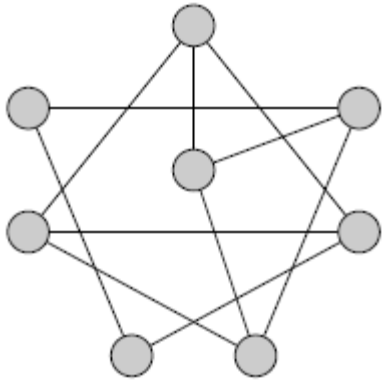
Relating Problem Difficulty with Reductions

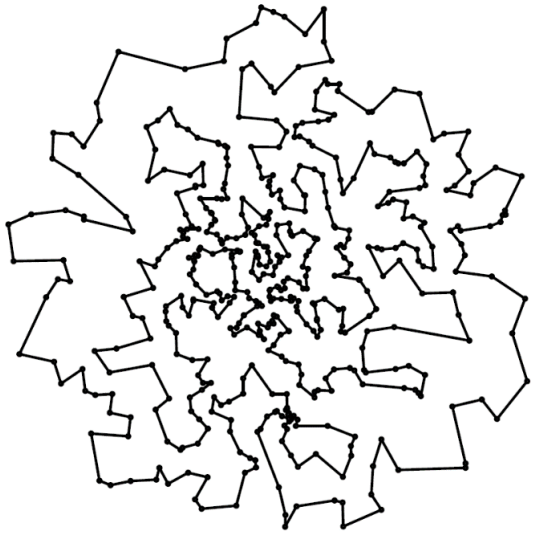
- Reductions between problems let you transfer hardness results.

- Recall that element uniqueness (testing if n elements are all distinct) has an Ω(n log n) worst-case lower bound in the comparison model.
- Finding the most frequently occurring element must also take Ω(n log n) time in the worst case in the comparison model, since we can reduce element uniqueness to this problem.

Relating Problem Difficulty with Reductions

- Hamiltonian cycle: determine if a graph has a so-called “Hamiltonian” cycle, visiting every node exactly once.
- Traveling salesman: given the pairwise distances between N cities, find a minimum-length tour visiting them all exactly once.
- Hamiltonian cycle reduces to traveling salesman.
- If we could solve traveling salesman in polynomial time, we could also find Hamiltonian cycles in polynomial time.



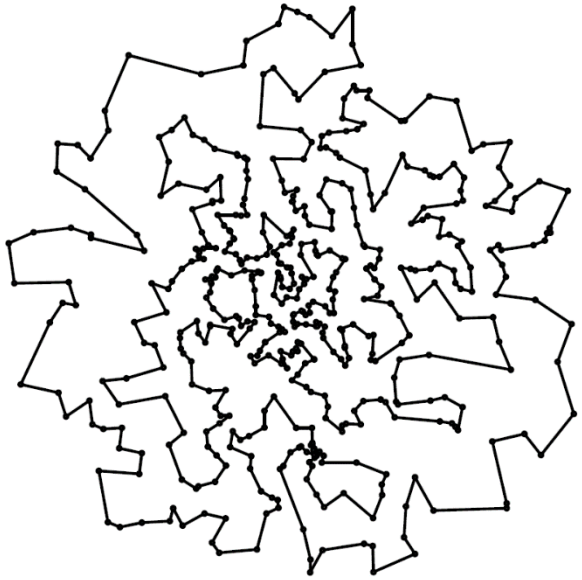


Bad News: NP-Hard Problems

- Prominent problem classes:
  - **P:** Yes/No problems solvable in polynomial time.
  - **NP:** Yes/No problems solvable in nondeterministic polynomial time.
  - **NP-Hard:** Problems for which existence of a polynomial-time solution would imply a polynomial-time solution for every problem in NP (meaning P = NP).
- We strongly suspect NP-hard problems cannot be solved efficiently, since NP contains thousands of problems we don't know how to solve efficiently.
- However, nobody has been able to prove this yet, and the "P vs. NP" problem is still probably the most famous open problem in all of computer science!

What to do with Hard Problems

- Common adage: showing a problem is NP-hard is a respectable way to explain to your boss why you aren't able to make algorithmic headway towards solving it efficiently.



What to do with Hard Problems

- As a college student, you can pick only 2 of these:
  1. Good grades
  2. Sleep
  3. Strong social life

- When faced with a hard problem, pick 2:
  1. Ability to find an optimal solution
  2. Ability to solve every input instance
  3. Ability to devise an efficient algorithm